

FOODFLOW 4.1

Cloud-Based Multi-Vendor Food Ordering SaaS

Executive Summary: This report details the architectural design, full-stack development, and cloud deployment of **FOODFLOW 4.1**, an enterprise-grade, multi-vendor food ordering Software-as-a-Service (SaaS) ecosystem. Bridging the gap between academic theory and practical software engineering, this project demonstrates end-to-end full-stack development. The platform connects customers, local restaurant vendors, and platform administrators through dedicated, isolated portals. It integrates secure token-based authentication, real-time WebSocket state synchronizations, Google OAuth identity federation, automated transaction emails, and the Razorpay payment gateway. This report outlines the system requirements, architectural patterns, technical implementation details, major features (including a specialized Cash on Delivery flow and dynamic invoicing), technical challenges overcome, and internship learning outcomes.

Submitted By: Sreeharan M Anilkumar

Organization: Bairuha Tech

Register No: MEC23AIM006

Mentors: Ambika Raj, Ressaan

Program: B.Tech, Artificial Intelligence & ML

Date: June 2026

1. Introduction & Problem Domain

1.1 Problem Statement

Traditional local food ordering systems suffer from fragmented user workflows, lack of scalability, and limited analytical capabilities. Small local restaurants and independent kitchens often lack the capital to invest in affordable, specialized software to manage online ordering, menu configuration, and sales auditing. Customers face disjointed order-tracking experiences, while platform owners struggle to govern vendor onboarding, enforce marketplace policies, or generate dynamic transactional documents.

1.2 Project Objectives

To address these bottlenecks, the objectives of this internship project were to:

- **Build a Scalable Multi-Vendor Platform:** Design a decoupled, multi-tenant architecture allowing independent restaurant vendors to manage separate menus and orders.
- **Provide Role-Based Portals:** Construct isolated user experiences for Customers, Restaurant Admins (Vendors), and Super Administrators.
- **Enforce Security & Data Isolation:** Implement secure authentication (JWT with refresh rotation) and row-level multi-tenant database isolation.
- **Enable Real-Time Operations:** Integrate WebSockets for live status updates and alerts on active orders.
- **Integrate Transactional Systems:** Embed secure payments (Razorpay), automated email notifications (Resend), and dynamic invoicing.

2. Technology Stack & Architecture

FOODFLOW 4.1 utilizes a decoupled client-server architecture. The system is designed to scale independently, maintaining separation of concerns between presentation, business logic, and storage layers.

Layer	Technologies	Role / Rationale
Frontend	Next.js 16/15 (App Router), React 19, TypeScript, Tailwind CSS v4, Framer Motion	Deliver a fast, responsive, and SEO-optimized UI with smooth transitions.
Backend	NestJS 11, Node.js, Socket.IO, Passport.js	Provide a modular, structured gateway and WebSocket server.
Database	PostgreSQL 16 (Neon Cloud), Prisma ORM	Ensure relational storage, type safety, and transactional control.
Security	JWT (Token Rotation), Google OAuth 2.0	Enforce secure stateless auth and social identity federation.
Integrations	Razorpay SDK, Resend API, Cloudinary	Handle checkout checkout payments, emails, and asset CDNs.

3. System Implementation & Deliverables

3.1 Database Design

The database structure leverages a highly normalized PostgreSQL schema. The core entities include: **User** (distinct roles: *CUSTOMER*, *ADMIN*, *SUPER_ADMIN*), **Restaurant** (onboarding states), **Food & Category** (hierarchical menus), **Order & OrderItem** (purchases snapshot), **Payment & Invoice** (financial audits), and **AuditLog** (system-wide administrative tracking).

3.2 System Modules

Module A: The Customer Portal - Tailored to discovery and checkout. Includes horizontally scrollable categories, smart cart coupon validation, and WebSocket-driven live order tracking timeline updates.

Module B: The Restaurant Admin Console - A vendor dashboard with multi-tenant isolation. Includes live incoming order queues, menu builders, and Recharts-based 7-day revenue trend visualizations.

Module C: The Super Admin Control Center - The central command console. Tracks gross platform sales, active merchants, registered user records, and logs system-wide administrative audit actions.

4. Key Engineering Deliverables

4.1 Cash on Delivery (COD) Razorpay Bypass

To accommodate door-step cash payments without database migrations, we implemented a bypass: 1.

Frontend Integration: Next.js intercepts COD selections, bypasses Razorpay SDK script overlay execution, and routes directly to the tracking screen. 2. **Backend Logic:** NestJS skips payment initialization, storing the order state as PENDING immediately. 3. **Data Integrity:** A zero-record design pattern was utilized, skipping payment row instantiation for COD orders to preserve database structure.

4.2 Dynamic Invoicing & Lifecycle Completion

COD orders complete their lifecycle as follows: 1. **Status Capture:** When the store admin updates the order status to DELIVERED, the backend flags paymentStatus to PAID. 2. **PDF Build:** InvoiceService dynamically generates a receipt showing Cash on Delivery as the method, and N/A for transaction ID. 3. **Mailing:** The receipt is sent immediately as an email attachment to the customer via the Resend API.

5. Technical Challenges & Solutions

- **Timezone-Safe Analytics:** Solved chart boundary discrepancies by standardizing database timestamps to UTC and formatting client-side graphs relative to local user location.
- **Tenant Data Isolation:** Created NestJS Guards that cross-verify authenticated JWT claims against request path parameters, preventing vendors from accessing another merchant's financial logs.
- **Node Environment PATH:** Resolved path executable conflicts on the host by defining explicit system shell runtime path profile variables.

6. Internship Learning Outcomes & Conclusion

6.1 Practical Skills Acquired

- **Enterprise Architecture:** Mastered NestJS modular structure, dependency injection, and Prisma client transactions.
- **Real-Time System Design:** Handled WebSocket channel room management, Socket.IO clients, and cache invalidation.
- **Integrations & Security:** Integrated Razorpay SDK checkout scripts, Google OAuth federation, and Resend mail APIs.
- **Operations & Delivery:** Conducted database schema migrations, environment isolation, and Railway/Vercel continuous integration pipelines.

6.2 Conclusion

The Bairuha Tech internship successfully delivered FOODFLOW 4.1 as a robust, scalable, multi-vendor SaaS platform. Taking the codebase from concept to fully synchronized cloud deployment deepened my understanding of web engineering, data security, and transactional reliability. I am confident that these engineering insights will serve as a strong foundation for my future career in full-stack cloud systems engineering.